

ARISTOTLE UNIVERSITY OF THESSALONIKI
FACULTY OF ENGINEERING
SCHOOL OF MECHANICAL ENGINEERING

Data Analysis

5th ASSIGNMENT

CHARITOPOULOS KYRIAZIS

AEM: 7137

INSTRUCTOR:

SOFIA PANAGIOTIDOU, Associate Professor

THESSALONIKI, MAY 2026

Note: tables and figures are referenced by their original labels (e.g. “Image 1”). Insert your own outputs at the marked placeholders.

Introduction

The subject of the present, fifth in order, assignment is the development, training and comparative evaluation of two different Neural Network architectures, with the aim of automatically classifying images of handwritten digits. For this purpose, the MNIST dataset is used, which is a benchmark in the field of computer vision. The set includes grayscale images of dimensions 28x28 pixels, depicting the digits on the scale from 0 to 9. Through this assignment, a simple, Fully Connected Neural Network (Simple NN) will be compared with a Convolutional Neural Network (Convolutional NN), examining their accuracy, the number of their parameters, as well as their resistance to the phenomenon of overfitting.

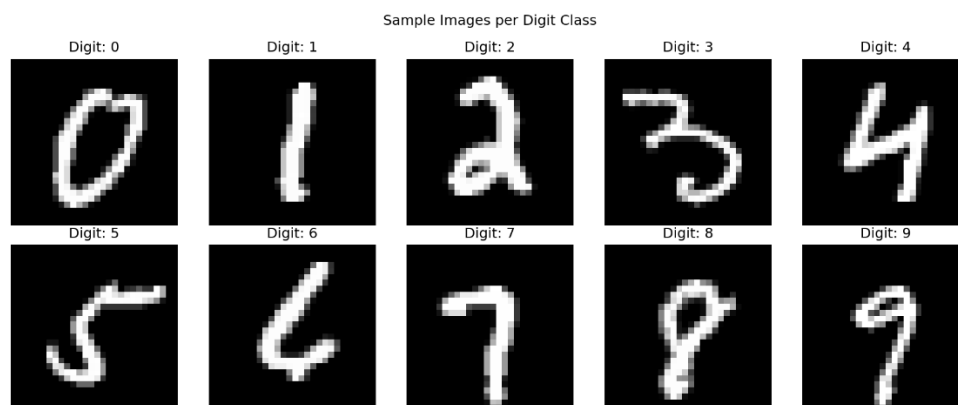


Image 1

Data Preparation and Preprocessing

The first and most decisive step in the development of any machine learning model is the correct preparation of the data. Based on the requirements, the data were loaded from the CSV files (labels.csv). As a “seed” to ensure the reproducibility of the random sampling, the AEM: 7137 was used. 90% of the original training set was selected, creating a Train Set of 54,000 samples (out of the original 60,000), and 80% of the original test set, creating a Test Set of 8,000 samples (out of the original 10,000).

```
Original train set size : 60,000 samples
Original test set size : 10,000 samples
Subsampled train set    : 54,000 samples (90%)
Subsampled test set     : 8,000 samples (80%)
```

Image 2

Class Balance:

Before training, it was checked whether the distribution of the digits in the Train Set is balanced. As depicted in the distribution graph (Image 3), the data subset has from approximately 4,800 to 6,000 samples per class. This balance is of vital importance, as it ensures that the network will not develop a statistical bias in favor of the most frequently appearing class during training.

```
Samples per digit in train set:  
Digit 0: 5,340  
Digit 1: 6,094  
Digit 2: 5,335  
Digit 3: 5,473  
Digit 4: 5,274  
Digit 5: 4,858  
Digit 6: 5,320  
Digit 7: 5,671  
Digit 8: 5,266  
Digit 9: 5,369
```

Image 3

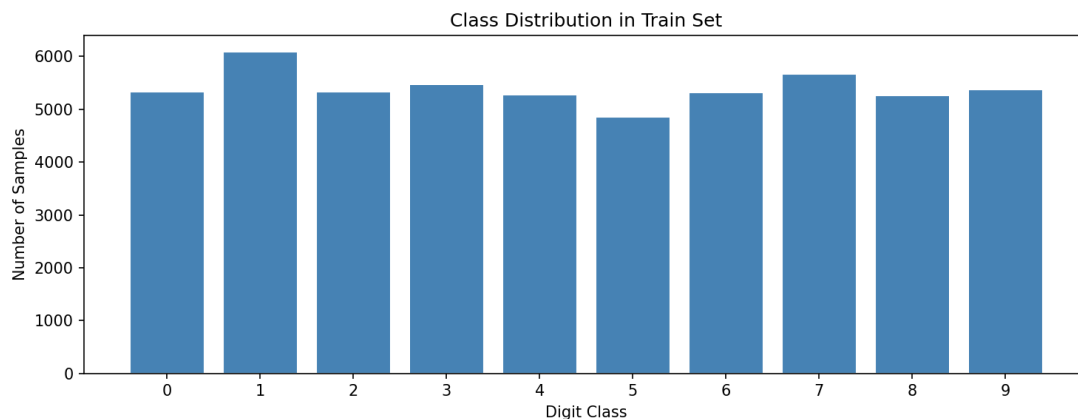


Image 4

Normalization:

The original values of the pixels in a grayscale image range from 0 (absolute black) to 255 (absolute white). In the code, a division of the entire input matrix by 255.0 was performed ($X_{\text{train}} / 255.0$), transforming the values into the interval $[0.0, 1.0]$. This process is crucial for the smooth and rapid convergence of the optimization algorithm. When the input values are large or non-uniform, the gradients during backpropagation may become unstable, delaying the training or trapping the model in local minima.

Flattening:

For the needs of the Simple Neural Network, the two-dimensional structure of the images (28x28) had to be modified. Using the appropriate function, the images were converted into one-dimensional vectors (1D) with a length of 784 elements ($28 \times 28 = 784$). This is required because the fully connected (Dense) layers accept exclusively one-dimensional inputs, treating each pixel as an independent variable-feature, temporarily ignoring their spatial correlation.

SECTION A: Simple Neural Network (Simple NN)

Architecture and Training:

In the first section, a Sequential model was developed, consisting exclusively of Dense Layers. The architecture that was implemented, based on the specifications, is the following:

- **Input Layer:** 784 neurons (one for each pixel of the Flattened image) with a ReLU activation function.
- **Hidden Layer 1:** 256 neurons, ReLU activation.
- **Hidden Layer 2:** 128 neurons, ReLU activation. The use of ReLU (Rectified Linear Unit) introduces the necessary non-linearity into the network, allowing it to learn complex relationships.
- **Output Layer:** 10 neurons, Softmax activation. Softmax converts the outputs of the 10 neurons into a probability distribution, where their sum equals 1, indicating the probability that the image belongs to each one of the 10 digits.

The model was configured with the adam optimizer, which was chosen because it automatically adjusts the learning rate per parameter, ensuring faster and more stable convergence. As the loss function, `sparse_categorical_crossentropy` was defined, which is the appropriate one for multi-class classification problems where the labels are given as integer numbers. The training lasted 30 epochs, with 10% of the data kept for Validation. The total number of trainable parameters of the network amounted to 850,586.

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 784)	615,440
dense_1 (Dense)	(None, 256)	200,960
dense_2 (Dense)	(None, 128)	32,896
dense_3 (Dense)	(None, 10)	1,290

Total params: 850,586 (3.24 MB)

Image 5

Evaluation and Discussion of Results:

The evaluation of the trained model on the Test Set yielded an Accuracy of 98.47% and a Loss of 0.0896. Studying the training curves (Image 6), a characteristic phenomenon of overfitting is observed. More specifically, in the Loss graph (right), while the Train Loss continues to decrease, reaching a minimum of 0.0056 at the 30th epoch, the Validation Loss stops decreasing around the 5th epoch and then presents an upward trend, closing at 0.1650. This proves that the model, due to its large capacity (many parameters), began to memorize the particularities and the “noise” of the Train Set, partly losing its generalization ability on new data.

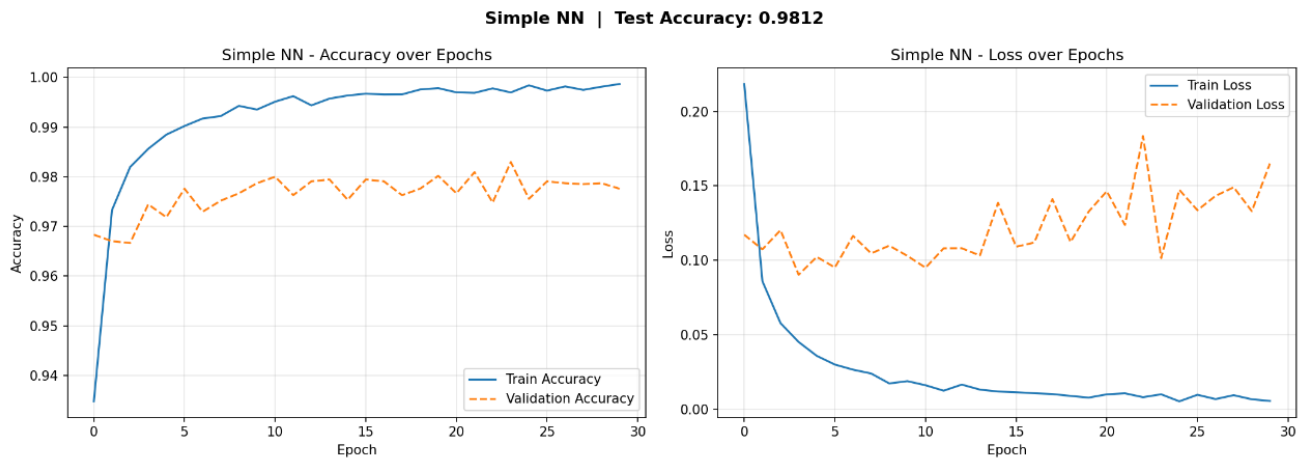


Image 6

Image 7 presents the Confusion Matrix. Examining the elements outside the main diagonal (the model's errors), we identify specific patterns. According to the code analysis, the most frequent errors (Most confused digit pairs) were:

1. Actual 9 -> Predicted as 4 (17 times).
2. Actual 7 -> Predicted as 2 (10 times).
3. Actual 3 -> Predicted as 7 (6 times).

These errors are fully explained based on the morphology of the digits. The handwritten '9' (when the upper loop does not close well) has an almost identical structure to the handwritten '4' (a vertical line and lines in the upper-left quadrant). The fact that the Dense network processes the image as a one-dimensional 1D vector (Flattened) makes it extremely vulnerable to small shifts of the pixels, being unable to perceive the overall "image" and the spatial relationship of the features.

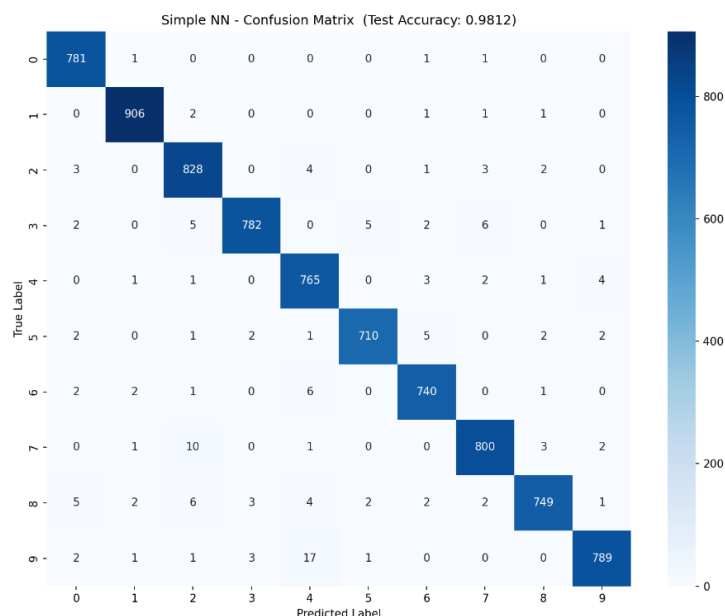


Image 7

SECTION B: Convolutional Neural Network (CNN)

Architecture and Training:

In the second section, an attempt was made to address the weaknesses of the Simple NN by exploiting the 2D spatial structure of the images. For this purpose, the data were reshaped by adding a fourth dimension (batch_size, 28, 28, 1) corresponding to the single channel of the grayscale images.

The architecture of the CNN was structured as follows:

Conv2D Layer: 32 filters of size (3x3), ReLU activation. These filters "scan" the image, locating basic features (edges, curves) independently of their position.

MaxPooling2D: A (2x2) window, which halves the dimensions of the feature maps, keeping only the most intense information (max value) and drastically reducing the computational load.

Conv2D Layer: 64 filters (3x3), ReLU activation. Here the network composes the simple features of the previous layer in order to locate more complex shapes (e.g. closed loops, corners).

Flatten & Dense: The 2D features are flattened (Flatten) and feed a hidden Dense layer of 64 neurons (ReLU), before ending up at the Output Layer of 10 neurons (Softmax).

The training was carried out with exactly the same settings (adam, 30 epochs, 10% validation) so that the comparison is correct. It is noteworthy that the CNN, despite the complexity of the operations, had only 515,146 trainable parameters.

This is the same passage that's already in your Assignment 5 report (the CNN architecture section), so it's consistent with what's in those files if you need to cross-reference.

The training was carried out with exactly the same settings (adam, 30 epochs, 10% validation) so that the comparison is correct. It is noteworthy that the CNN, despite the complexity of the operations, had only 515,146 trainable parameters.

Model: "CNN"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
flatten (Flatten)	(None, 7744)	0
dense_4 (Dense)	(None, 64)	495,680
dense_5 (Dense)	(None, 10)	650

Total params: 515,146 (1.97 MB)

Image 8

CNN Evaluation:

The performance of the CNN on the Test Set was superior, achieving an Accuracy of 99.21% and a reduced Loss of 0.0425. Examining its own training curves (Image 9) and the confusion

matrix (Image 10), we find that the matrix is much “cleaner” (a stronger main diagonal) and that the characteristic confusions of the Simple NN (e.g. 9 with 4) have been significantly reduced.

Image 10 presents the Confusion Matrix for the CNN model. According to the code analysis, the most frequent errors were:

- 4. Actual 4 -> Predicted as 9 (14 times).
- 5. Actual 2 -> Predicted as 7 (13 times).
- 6. Actual 8 -> Predicted as 6 (7 times).

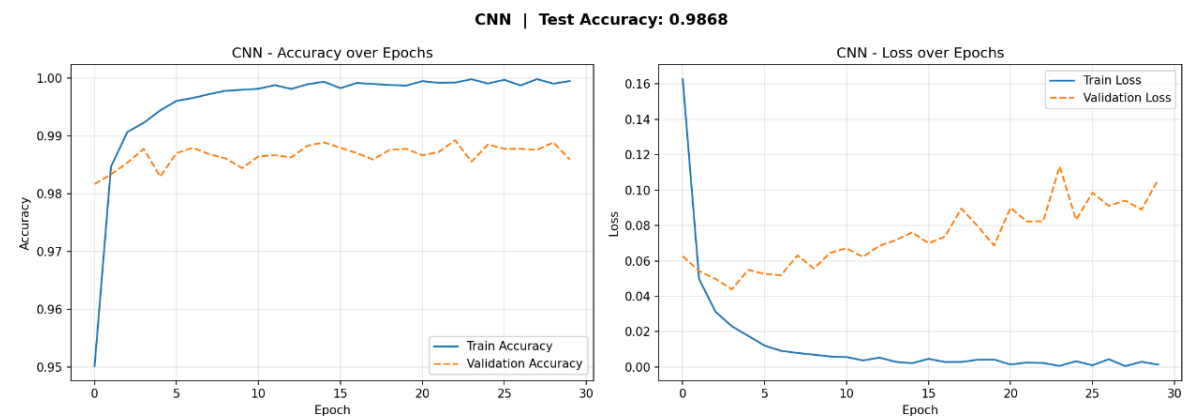


Image 9

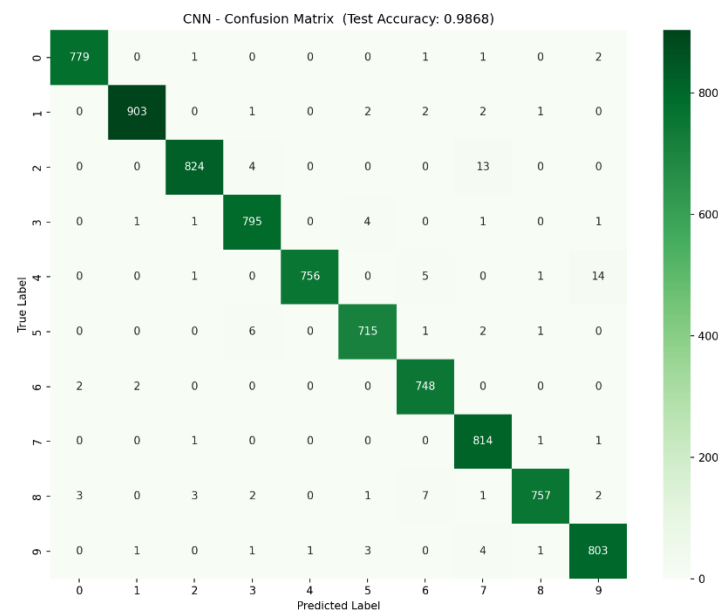


Image 10

Comparison and Final Conclusions

The final stage of the analysis concerns the direct comparison of the two models. Their overall performance is depicted visually in the comparative diagram of Image 11 (Validation Performance).

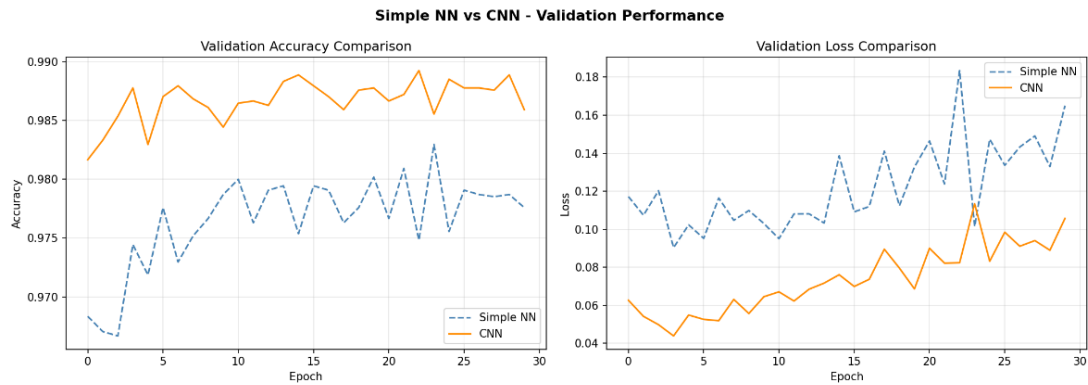


Image 11

The conclusions that are drawn are multidimensional:

Addressing Overfitting: In the right graph of Image 11, the advantage of the CNN becomes clear. While the Validation Loss of the Simple NN (blue dashed line) diverges upward after the 10th epoch (an indication of overfitting), the corresponding line of the CNN (orange) remains much flatter and more stable throughout the training. The CNN demonstrated a superior ability to generalize.

Accuracy and Pattern Recognition Ability: The CNN surpassed the Simple NN in recognition ability (99.21% vs 98.47%). This is due to the fact that Convolutional NNs are specifically designed for images. In contrast to the Simple NN, which needs to see a feature (e.g. a corner) in the exact position where it learned it, the CNN possesses translation invariance. Its filters scan the entire surface, locating the morphological features wherever they are.

Parameter Reduction: The most impressive finding comes from the `.summary()` analysis. The Simple NN required 850,586 parameters, while the CNN only 515,146. That is, a parameter reduction of the order of 39.4% was achieved, with a simultaneous increase in performance! This "paradox" is due to the Weight Sharing mechanism. In a Dense network, each pixel is connected to a separate neuron with its own weight. In the CNN, the 9 weights of a 3x3 filter are applied repeatedly over the entire image, saving the need for hundreds of thousands of separate connections.

This is also part of your Assignment 5 report's "Comparison and Final Conclusions" section, so it matches what's already in those translated files.

Metric	Simple NN	CNN
Test Accuracy	0.9847	0.9921
Test Loss	0.0896	0.0425
Trainable Parameters	850,586	515,146
Parameter Reduction	---	39.4%

Image 12

In conclusion, Convolutional Neural Networks confirm the theory that holds them to be the optimal architecture for computer vision problems, offering higher accuracy, lower computational cost and greater resistance to overfitting.